

An Internship Report
OF
ENTERPRISE CRM & E-COMMERCE MANAGEMENT SYSTEM

A Full-Stack Web Application for Customer and Business Data Management

Submitted
To
School of Computer Science and Engineering
IILM University, Greater Noida, Uttar Pradesh
In partial fulfilment of the requirement of degree of
B.Tech (Computer Science & Engineering)



By
Ashish Yadav
Roll No.: 25SCS1003005067
Section: 2CSE32
Session: 2025-2029
Course: B.Tech CSE

Academic Session 2025-2029

CANDIDATE'S DECLARATION

I, Ashish Yadav, do hereby declare that the project report titled “ENTERPRISE CRM & E-COMMERCE MANAGEMENT SYSTEM” has been completed by me to fulfil the academic requirements for the degree of Bachelor of Technology in Computer Science and Engineering. The work presented in this report describes the full-stack web application developed for the project.

I affirm that this report is prepared for academic evaluation and that the technologies, frameworks, and services used in the implementation are acknowledged in the appropriate sections. All the references have been quoted and I have not taken as such any material from any other source. I further declare that this report has not been submitted to any other institute for any degree/diploma requirement, and I shall be solely responsible for any kind of copyright violation in this regard.

Student Name: Ashish Yadav
Roll No.: 25SCS1003005067

ACKNOWLEDGEMENT

I am using this opportunity to express my sincere gratitude to everyone who supported me throughout the development of this project. The project provided practical exposure to frontend development, backend API design, database integration, authentication, environment configuration, testing, and debugging.

I am thankful to IILM University, Greater Noida, and the faculty of the School of Computer Science and Engineering for providing an academic environment in which I could apply software-development concepts to a practical full-stack application.

I also express my gratitude to my parents, friends, and everyone who encouraged me during the development and documentation of this project.

Student Name: Ashish Yadav

Roll No.: 25SCS1003005067

TABLE OF CONTENTS

1. Introduction	4
1.1 Background and Motivation.....	4
1.2 Problem Statement	4
1.3 Objectives.....	4
1.4 Scope and Limitations	5
2. Literature Review and Related Work	6
3. System Requirements and Feasibility	7
3.1 Functional Requirements	7
3.2 Non-Functional Requirements.....	7
3.3 Feasibility Analysis	8
4. System Architecture and Design	9
4.1 High-Level Architecture.....	9
4.2 Use-Case Model	10
4.3 Data-Flow Diagram.....	11
4.4 Sequence of Application Calls	12
4.5 Component Diagram	13
5. Technology Stack.....	14
6. Implementation Details.....	15
6.1 Frontend Implementation.....	15
6.2 Backend API Implementation.....	15
6.3 Database Integration	15
6.4 Authentication and Registration	16
7. User Interface Design	17
8. Data Management and Application Modules	18
9. Testing and Performance Evaluation	19
10. Results and Discussion	20
11. Conclusion and Future Work.....	21
References	22
Appendix A – Development Environment.....	23
Appendix B – Final Screenshot Checklist.....	23

LIST OF FIGURES

Figure 1.1 System Architecture Diagram

Figure 4.1 High-Level Architecture

Figure 4.2 Use-Case Model

Figure 4.3 Level-1 Data-Flow Diagram

Figure 4.4 Sequence of Application Calls

Figure 4.5 Component Diagram

Figure 7.1 Registration Interface / User Flow

LIST OF TABLES

Table 3.1 Functional Requirements Summary

Table 3.2 Non-Functional Requirements

Table 5.1 Technology Stack Summary

Table 8.1 Application Module Summary

Table 9.1 Test Case Summary

1. INTRODUCTION

Modern organizations increasingly require web applications that bring customer information, business workflows, and commerce-related data into one manageable platform. Customer Relationship Management (CRM) systems help organizations maintain structured customer information and business interactions, while e-commerce capabilities support digital product and order workflows.

The present project, Enterprise CRM & E-Commerce Management System, is developed as a full-stack web application. The application separates the frontend from the backend and uses MongoDB Atlas as the cloud database. The architecture demonstrates the complete path from a browser-based interface to server-side API processing and persistent database storage.

The project was developed using a Vite-based frontend and a Node.js/Express backend. The backend was configured to run on port 5000, while MongoDB Atlas was used for remote database connectivity.

System Architecture - Enterprise CRM & E-Commerce Management System



Figure 1.1 System Architecture Diagram

1.1 Background and Motivation

A full-stack CRM/e-commerce application provides practical software-development experience because it requires coordination between the user interface, HTTP APIs, authentication, database models, error handling, and environment configuration. The motivation is to create a modular application that can be extended with additional enterprise functions without placing all logic in a single file.

1.2 Problem Statement

Businesses need a centralized system for maintaining customer and commerce-related information. Disconnected interfaces and manual data handling can make information difficult to maintain and update. The proposed system provides a browser-based interface connected to a backend API and persistent MongoDB database, creating a foundation for centralized CRM and commerce management.

1.3 Objectives

- Develop a clean full-stack web application with separate frontend and backend layers.
- Create backend REST APIs using Node.js and Express.

- Integrate MongoDB Atlas for persistent application data.
- Implement a user registration and authentication-related workflow.
- Organize backend logic using routes, controllers, models, middleware, and utilities.
- Provide a foundation for customer, product, and order management modules.
- Test the complete browser → API → database workflow.

1.4 Scope and Limitations

The project scope includes the frontend interface, Node.js/Express backend, MongoDB Atlas connectivity, environment-based configuration, registration/authentication flow, modular backend organization, and testing of the development setup. The exact business operations available in the final build depend on the routes, controllers, and models implemented in the project.

2. LITERATURE REVIEW AND RELATED WORK

CRM and e-commerce applications are established categories of business software. CRM platforms generally focus on customer information, interactions, and business processes, while e-commerce systems focus on products, orders, and digital transactions. Modern web architectures commonly separate presentation, application logic, and data storage so that individual layers can evolve independently.

REST-style APIs provide a practical communication boundary between browser applications and server-side services. Node.js is widely used for JavaScript-based backend development, while Express provides routing and middleware capabilities for HTTP applications. MongoDB is a document-oriented database suitable for flexible JSON-like business records.

Cloud database platforms such as MongoDB Atlas reduce the need to manage database infrastructure locally during development. Environment variables are used to separate configuration and credentials from application source code. Vite provides a fast development workflow for modern frontend applications.

The related work therefore supports a layered architecture for this project, with clear boundaries between user interaction, API processing, business logic, and data storage.

3. SYSTEM REQUIREMENTS AND FEASIBILITY

3.1 Functional Requirements

ID	Functional Requirement	Priority
FR-01	Provide a browser-based user interface.	High
FR-02	Allow users to create an account through registration.	High

ID	Functional Requirement	Priority
FR-03	Send frontend requests to backend APIs.	High
FR-04	Run backend services using Node.js and Express.	High
FR-05	Store and retrieve application data using MongoDB Atlas.	High
FR-06	Organize server functionality into routes/controllers/models/middleware.	High
FR-07	Return meaningful success or error responses.	High
FR-08	Support independent local execution of frontend and backend.	Medium

Table 3.1 Functional Requirements Summary

3.2 Non-Functional Requirements

Attribute	Requirement
Performance	Application interactions should provide practical response times in local development.
Usability	Forms and pages should be understandable and provide clear feedback.
Reliability	Database/API failures should be handled and reported clearly.
Security	Database credentials should be kept in environment variables and not exposed to the client.
Maintainability	Frontend and backend responsibilities should remain separated and backend modules organized logically.
Compatibility	The application should run in a modern web browser and supported Node.js environment.

Table 3.2 Non-Functional Requirements

3.3 Feasibility Analysis

Technical Feasibility: React/Vite, Node.js, Express, and MongoDB Atlas form a practical stack for a full-stack academic application.

Operational Feasibility: The application can be developed locally by running frontend and backend as separate processes.

Economic Feasibility: Development can be performed using commonly available open-source tools and a development-oriented MongoDB Atlas deployment.

4. SYSTEM ARCHITECTURE AND DESIGN

4.1 High-Level Architecture

The system follows a layered client-server architecture. The browser provides the presentation layer, the Node.js/Express application provides the API and business-logic layer, and MongoDB Atlas provides persistent data storage.

System Architecture - Enterprise CRM & E-Commerce Management System

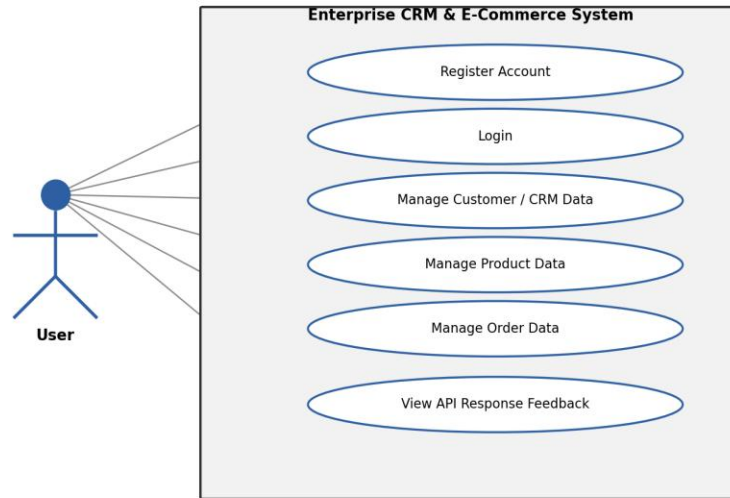


Figure 4.1 High-Level System Architecture – Layer Summary

Layer	Main Components	Responsibility
Presentation	React + Vite	User interface, forms, pages, feedback and navigation.
Application	Node.js + Express	HTTP requests, routing, middleware and API responses.
Business Logic	Controllers / Utilities	Request processing and application operations.
Data Access	Models / Mongoose	Database-facing data structures and operations.
Storage	MongoDB Atlas	Persistent application records.

4.2 Use-Case Model

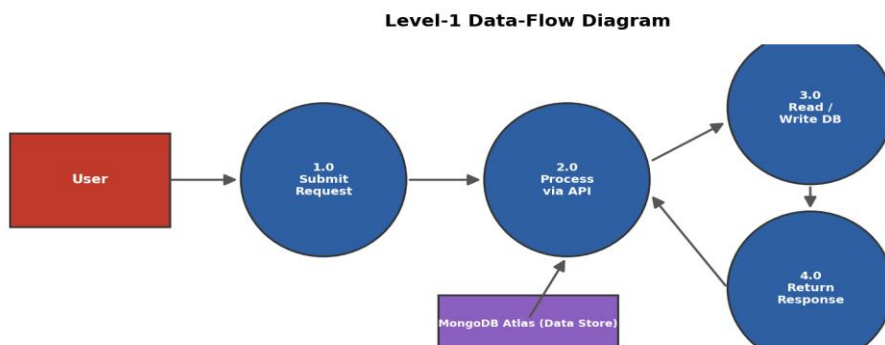
The primary actor is the application user. The main use cases are account registration/login and business-oriented operations such as customer, product, and order management where those modules are implemented.

Use-Case Diagram - CRM & E-Commerce System*Figure 4.2 Use-Case Model – Interaction Summary*

Actor	Use Case	System Interaction
User	Register	Frontend → API → Database → Response
User	Login	Frontend → Authentication API → Response
User	Manage CRM data	Frontend → API → Database → Response
User	Manage commerce data	Frontend → API → Database → Response

4.3 Data-Flow Diagram

A typical data flow begins with a user action in the browser. The frontend sends an HTTP request to the Express API. Routes direct the request to controllers and supporting logic, which communicate with the database layer. The result is returned to the browser.

*Figure 4.3 Level-1 Data-Flow Diagram – Flow Summary*

Flow	Description
1. User Input	User enters data or selects an operation in the browser.
2. API Request	Frontend sends an HTTP request to the backend.
3. Server Processing	Express routes request to application logic.
4. Database Operation	Models/database layer read or write MongoDB Atlas.
5. API Response	Backend returns success/error data.
6. UI Update	Frontend displays the result.

4.4 Sequence of Application Calls

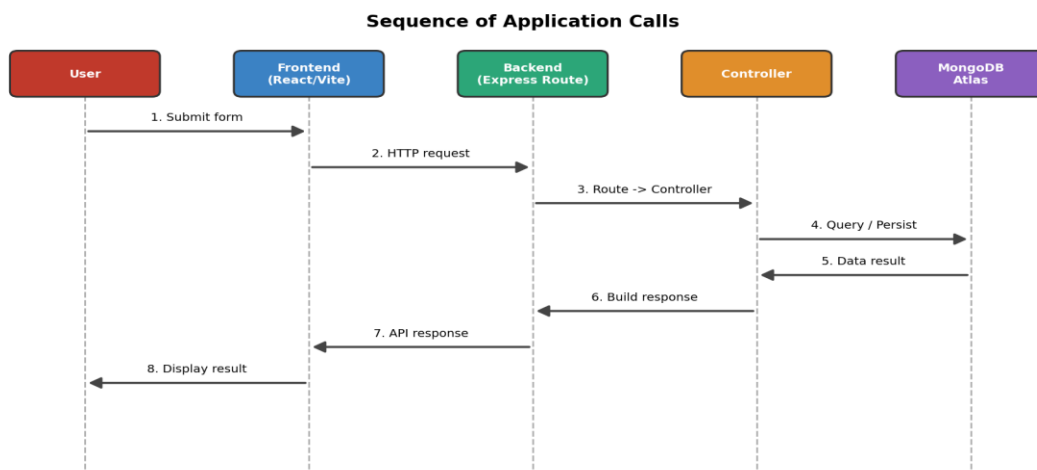
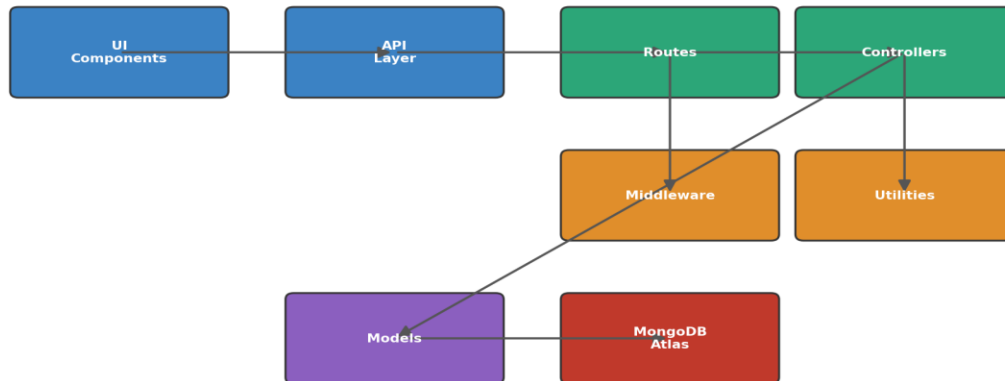


Figure 4.4 Sequence of Application Calls

Step	Actor	Operation
1	User	Submit registration/data form.
2	Frontend	Create HTTP API request.
3	Backend	Receive request through Express route.
4	Controller	Validate/process request.
5	Database	Read/write MongoDB Atlas data.
6	Backend	Create API response.
7	Frontend	Display result to user.

4.5 Component Diagram

Component Diagram - Backend Organization*Figure 4.5 Component Diagram – Component Relationships*

Component	Connected To	Role
UI Components	API Layer	Collect input and display output.
API Layer	Routes / Controllers	Send and receive HTTP data.
Routes	Controllers	Map endpoints to server logic.
Controllers	Models	Perform business/data operations.
Middleware	Routes / Controllers	Authentication, validation, and common processing where implemented.
Utilities	Server Modules	Shared helpers and configuration.
Models	MongoDB Atlas	Database access.

5. TECHNOLOGY STACK

Technology choices balance developer productivity, ecosystem maturity, maintainability, and suitability for a full-stack academic application.

Layer	Technology	Purpose / Rationale
Frontend	React + Vite	Component-based interface and fast development workflow.
Language	JavaScript	Used for frontend and backend development.
Backend Runtime	Node.js	Server-side JavaScript runtime.
API Framework	Express.js	HTTP server, routing and middleware.
Database	MongoDB Atlas	Cloud-hosted NoSQL database.
ODM	Mongoose	MongoDB connection/object modeling where used.

Layer	Technology	Purpose / Rationale
Package Manager	npm	Dependency installation and project scripts.
Development	Nodemon	Automatic backend restart during development.
Configuration	.env	Environment-specific values such as MongoDB URI.

Table 5.1 Technology Stack Summary

5.1 Frontend

The frontend is maintained in a separate frontend directory and is developed using Vite. It provides the user-facing interface and communicates with the backend API.

5.2 Backend

The backend is maintained in a separate backend directory. Its structure contains configuration, controllers, middleware, models, routes, utilities, package files, and the server.js entry point.

5.3 Database

MongoDB Atlas provides the remote database deployment. The connection string is supplied to the backend through environment configuration.

6. IMPLEMENTATION DETAILS

6.1 Frontend Implementation

The frontend uses Vite for local development and provides account-related screens and application views. User actions are sent to backend API endpoints rather than directly accessing the database.

6.2 Backend API Implementation

The backend initializes an Express application, configures request middleware, enables CORS configuration, exposes a health endpoint, and establishes the MongoDB connection. Routes and controllers keep application logic modular.

6.3 Database Integration

MongoDB Atlas is connected through an environment variable containing the MongoDB URI. During development, configuration problems included an undefined URI, SRV/network connection refusal, and authentication failure. After correcting the environment and Atlas settings, the backend reported a successful MongoDB connection.

6.4 Authentication and Registration

The application includes an account-creation interface with fields for Full Name, Email, and Password. The intended workflow is frontend form submission → backend API → validation/processing → MongoDB persistence → response to the frontend.

For production use, passwords should be securely hashed, credentials should remain server-side, validation should be applied on the server, and protected routes should require appropriate authentication.

7. USER INTERFACE DESIGN

The user interface acts as the entry point to the full-stack system. The verified registration screen contains Full Name, Email, and Password fields and an area for registration feedback.

7.1 User Flow

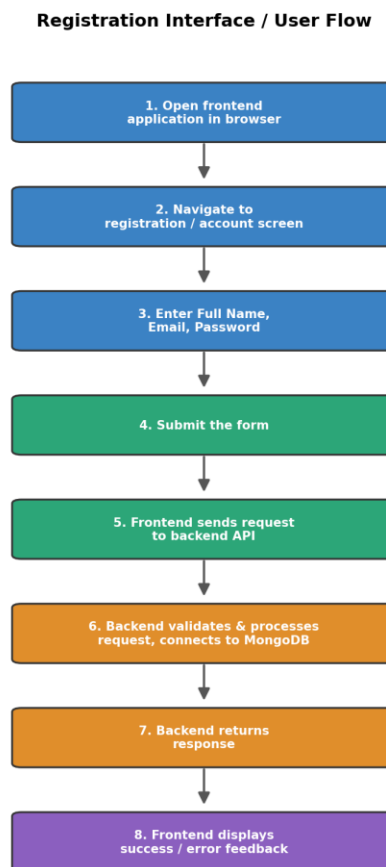


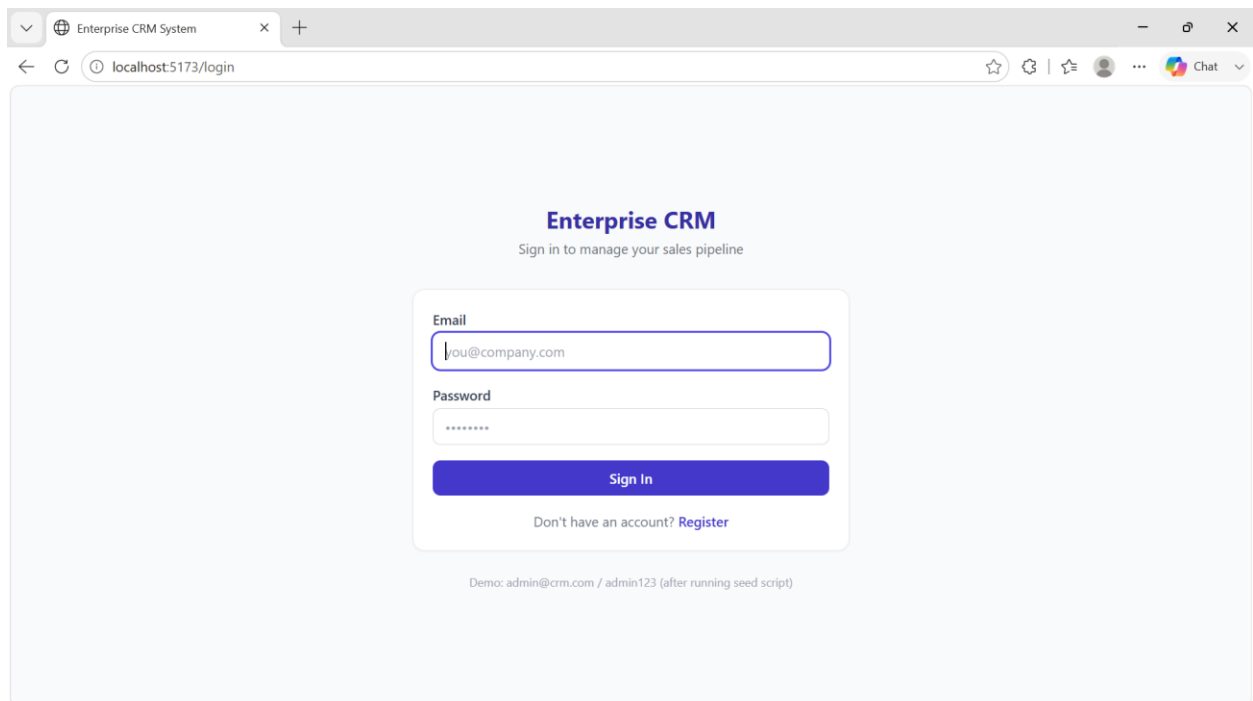
Figure 7.1 Registration Interface / User Flow

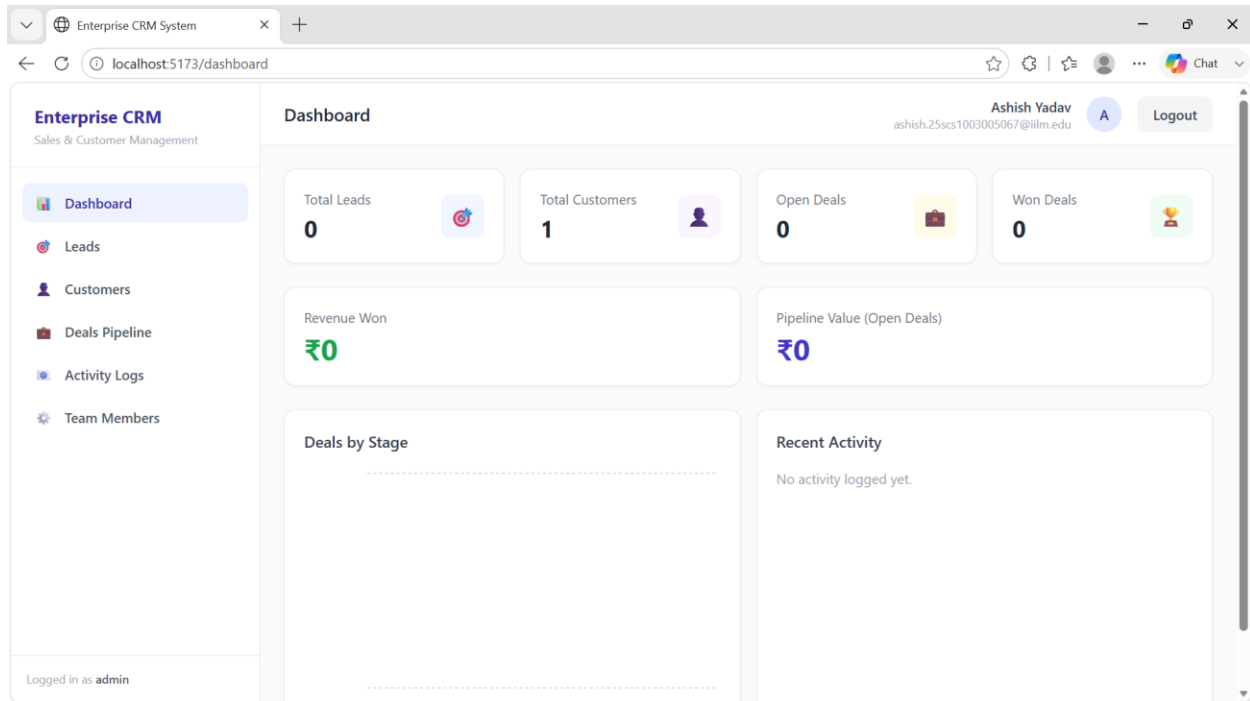
Step	Action
1	Open the frontend application in a browser.
2	Navigate to the registration/account screen.
3	Enter full name, email, and password.
4	Submit the form.
5	Frontend sends the request to the backend API.
6	Backend validates/processes the request and communicates with MongoDB.
7	Backend returns a response.
8	Frontend displays success/error feedback.

7.2 UI Design Principles

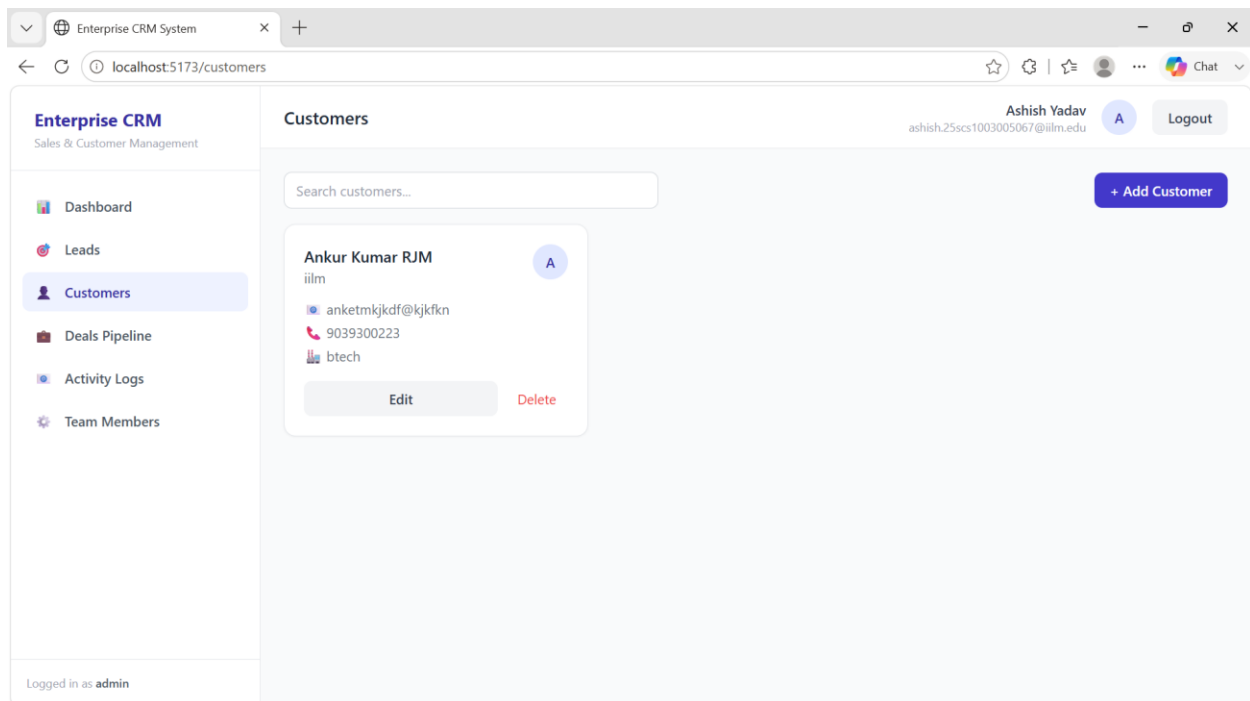
The interface should use clear labels, readable forms, understandable navigation, and useful feedback. Error messages should not expose database credentials, connection strings, or internal server details.

7.3 Screenshots





The screenshot shows the 'Enterprise CRM' dashboard. The left sidebar contains navigation links: Dashboard, Leads, Customers, Deals Pipeline, Activity Logs, and Team Members. The main content area is titled 'Dashboard' and displays several key metrics: Total Leads (0), Total Customers (1), Open Deals (0), and Won Deals (0). Below these are two larger cards: 'Revenue Won' showing ₹0 and 'Pipeline Value (Open Deals)' showing ₹0. At the bottom, there are two sections: 'Deals by Stage' and 'Recent Activity', both of which are currently empty. The top right corner shows the user 'Ashish Yadav' with a 'Logout' button. The bottom left corner indicates the user is logged in as 'admin'.



The screenshot shows the 'Enterprise CRM' 'Customers' page. The left sidebar is identical to the dashboard view. The main content area is titled 'Customers' and features a search bar labeled 'Search customers...'. A '+ Add Customer' button is located in the top right corner. Below the search bar, a customer card for 'Ankur Kumar RJM' is displayed. The card includes the email 'iilm', the phone number '9039300223', and the institution 'btech'. At the bottom of the card are 'Edit' and 'Delete' buttons. The top right corner shows the user 'Ashish Yadav' with a 'Logout' button. The bottom left corner indicates the user is logged in as 'admin'.


```

App.jsx
1 import React from "react";
2 import { Routes, Route, Navigate } from "react-router-dom";
3 import Login from "../pages/Login";
4 import Register from "../pages/Register";
5 import Dashboard from "../pages/Dashboard";
6 import Leads from "../pages/Leads";
7 import Customers from "../pages/Customers";
8 import Deals from "../pages/Deals";
9 import Activities from "../pages/Activities";
10 import Users from "../pages/Users";
11 import ProtectedRoute from "../components/ProtectedRoute";
12
13 function App() {
14   return (
15     <Routes>
16       <Route path="/login" element={<Login />} />
17       <Route path="/register" element={<Register />} />
18
19       <Route path="/dashboard" element={<ProtectedRoute><Dashboard /></ProtectedRoute>} />
20       <Route path="/leads" element={<ProtectedRoute><Leads /></ProtectedRoute>} />
21       <Route path="/customers" element={<ProtectedRoute><Customers /></ProtectedRoute>} />
22       <Route path="/deals" element={<ProtectedRoute><Deals /></ProtectedRoute>} />
23       <Route path="/activities" element={<ProtectedRoute><Activities /></ProtectedRoute>} />
24       <Route path="/users" element={<ProtectedRoute adminOnly><Users /></ProtectedRoute>} />
25     </Routes>
26   );
27 }
28
29 export default App;
  
```

Terminal Output:

```

ASUS@DESKTOP-VMSKHOF MINGW64 ~/Desktop/New folder/enterprise-crm/backend ...
GET /api/auth/users 304 56.356 ms - -
  
```

```

server.js
3 const cors = require("cors");
4 const morgan = require("morgan");
5 const connectDB = require("../config/db");
6 const { NotFound, errorHandler } = require("../middleware/errorMiddleware");
7
8 const authRoutes = require("../routes/authRoutes");
9 const leadRoutes = require("../routes/leadRoutes");
10 const customerRoutes = require("../routes/customerRoutes");
11 const dealRoutes = require("../routes/dealRoutes");
12 const activityRoutes = require("../routes/activityRoutes");
13 const dashboardRoutes = require("../routes/dashboardRoutes");
14
15 connectDB();
16
17 const app = express();
18
19 app.use(cors({ origin: process.env.CLIENT_URL || "*", credentials: true }));
20 app.use(express.json());
21 app.use(express.urlencoded({ extended: true }));
22 if (process.env.NODE_ENV !== "production") app.use(morgan("dev"));
23
24 app.get("/api/health", (req, res) => res.json({ status: "ok", message: "CRM API is running" }));
25
26 app.use("/api/auth", authRoutes);
  
```

Terminal Output:

```

ASUS@DESKTOP-VMSKHOF MINGW64 ~/Desktop/New folder/enterprise-crm/backend ...
GET /api/auth/users 304 56.356 ms - -
  
```

8. DATA MANAGEMENT AND APPLICATION MODULES

The project structure is suitable for enterprise-oriented data management. The exact data entities available in the final build depend on the implemented models and routes.

Module	Purpose	Implementation / Extension
User / Authentication	Registration, login and account access.	Core workflow
Customer / CRM	Customer-oriented records and business information.	CRUD extension
Product	Product and commerce information.	Extend where implemented
Order	Order workflow and status.	Future/extension if not present
Database	Persistent application storage.	MongoDB Atlas
API	Frontend-backend communication.	Core backend layer

Table 8.1 Application Module Summary

8.1 Data Security

Database credentials should be stored only in the backend environment configuration. Connection strings must not be published in repositories, screenshots, or frontend code. User passwords should be securely hashed in a production implementation.

8.2 Maintainability

The separation into routes, controllers, models, middleware, and utilities supports maintainability. New modules can be added without rewriting the entire server.

9. TESTING AND PERFORMANCE EVALUATION

Testing focused on the local development environment, frontend startup, backend startup, MongoDB connectivity, and registration flow.

ID	Test Case	Expected Result
TC-01	Install frontend dependencies	npm install completes successfully.
TC-02	Run frontend development script	Vite becomes ready.
TC-03	Run backend development script	Server listens on port 5000.
TC-04	Missing MongoDB URI	Clear configuration error is produced.
TC-05	SRV/DNS problem	Connection error identifies network/SRV issue.
TC-06	Incorrect database password	MongoDB returns authentication failure.

ID	Test Case	Expected Result
TC-07	Correct Atlas configuration	MongoDB connection succeeds.
TC-08	Open registration page	Registration form is displayed.
TC-09	Submit registration	Success or meaningful error feedback is displayed.

Table 9.1 Test Case Summary

9.1 Testing Strategy

The system was tested incrementally. Dependencies and development servers were checked first, then database connectivity, and finally browser-level behavior. This helps distinguish frontend, backend, database, and configuration failures.

9.2 Performance Evaluation

No fabricated latency or benchmark values are included. Formal performance evaluation should be performed on the final deployed application using a defined test environment and measured response times.

9.3 Observed Development Issues

The project encountered an undefined database URI, MongoDB SRV connection refusal, and authentication failure during setup. After configuration was corrected, the backend reported a successful MongoDB Atlas connection.

10. RESULTS AND DISCUSSION

The project demonstrates the intended full-stack pipeline: a Vite frontend communicates with a Node.js/Express backend, and the backend communicates with MongoDB Atlas for persistent storage.

The frontend development server reached a ready state. The backend ran on port 5000, and after database configuration was corrected, the backend reported a successful MongoDB Atlas connection.

The main outcomes are:

- Separate frontend and backend project structure.
- Working Node.js/Express backend development environment.
- Vite-based frontend development environment.
- MongoDB Atlas cloud database integration.
- Registration/account-creation interface.
- Environment-based database configuration.
- Modular backend organization using routes, controllers, models, middleware and utilities.

10.1 Discussion

A major practical outcome is the understanding that a full-stack application has multiple independent failure points. A frontend may load while the backend is unavailable, and the backend may run while database authentication or network access fails. End-to-end operation therefore requires every layer to be configured correctly.

The modular architecture provides a foundation for future CRM and e-commerce features while preserving separation between presentation, API, and database layers.

11. CONCLUSION AND FUTURE WORK

This report presented the design and development of the Enterprise CRM & E-Commerce Management System, a full-stack web application combining a Vite-based frontend, Node.js/Express backend, and MongoDB Atlas database.

The project provided practical experience in frontend-backend communication, REST-style API development, database integration, authentication-related workflows, environment configuration, modular server organization, and debugging.

11.1 Future Work

- Complete customer CRUD with search, filtering, and pagination.
- Add product and inventory management.
- Add order creation and order-status tracking.
- Implement role-based access for administrators, staff, and customers.
- Add dashboard analytics and reports.
- Strengthen authentication with secure hashing and protected routes.
- Add centralized validation, rate limiting, logging, and error handling.
- Deploy frontend and backend with secure environment variables.
- Add unit, integration, and end-to-end automated tests.

11.2 Final Conclusion

The project establishes a practical and extensible foundation for an enterprise-oriented web application. Its layered architecture makes the system easier to understand, test, maintain, and extend.

REFERENCES

- [1] Node.js Documentation – Server-side JavaScript runtime and application development.
- [2] Express.js Documentation – Routing, middleware, and HTTP application development.
- [3] MongoDB Documentation – Database concepts, documents, collections, and queries.
- [4] MongoDB Atlas Documentation – Cloud database deployment and connectivity.
- [5] Vite Documentation – Frontend development and build tooling.
- [6] npm Documentation – Package management and project scripts.
- [7] Mongoose Documentation – MongoDB object modeling for Node.js.

APPENDIX A – DEVELOPMENT ENVIRONMENT

Item	Project Value
Frontend Directory	frontend/
Backend Directory	backend/
Backend Entry Point	server.js
Frontend Tool	Vite
Backend Runtime	Node.js
Backend Framework	Express.js
Database	MongoDB Atlas
Package Manager	npm
Development Runner	Nodemon
Backend Port	5000
Configuration File	.env

APPENDIX B – FINAL SCREENSHOT CHECKLIST

- Registration page with form visible.
- Successful registration/login result.
- Main CRM/e-commerce dashboard.
- Customer/CRM module, if implemented.
- Product/inventory module, if implemented.
- Order module, if implemented.
- MongoDB Atlas collection/data view.
- Backend terminal showing successful server + database connection without credentials.